

Aurora Music: Cloud-Native Music Streaming and Audio Identification Platform

Reddim Thoshan Reddy

Dept. of Computer Science
Amrita Vishwa Vidyapeetham
India

bl.en.u4cse23247@bl.students.amrita.edu

S.J.L Vamsi

Dept. of Computer Science
Amrita Vishwa Vidyapeetham
India

bl.en.u4cse23249@bl.students.amrita.edu

Vundela Lasya

Dept. of Computer Science
Amrita Vishwa Vidyapeetham
India

bl.en.u4cse23258@bl.students.amrita.edu

Abstract

Aurora Music is a cloud-native audio streaming platform that has been created with Flask and Amazon Web Services (AWS). Users can stream audio from uploaded songs, create and edit playlists, save songs in their favs list, and perform audio identification of any song using audio data processed with Librosa. The project follows the principles of modern, distributed cloud architecture in order to enable a wide range of AWS services built upon: EC2, RDS, S3, IAM, ECR, CloudWatch, CloudFront, Systems Manager (SSM), Application Load Balancer (ALB), and Route 53. The application is packaged in Docker containers for consistent deployment across various environments. The Aurora Music application provides scalable storage for audio files, secure authentication, broadcasts media files with high efficiency, and provides centralized monitoring in accordance with coding standards necessary to comply with cloud security best practices.

Keywords: Flask, AWS, Docker, Cloud Computing, Music Streaming, Librosa, S3, EC2, RDS, Cloud-Native Application

1 Introduction

The need for scalable music streaming applications is at an all-time high, given the trend of increased cloud computing and online media consumption. Problems usually experienced with traditional web applications regarding scalability, storage management, deployment consistency, and securing resources will now be resolved due to the creation of Aurora Music, which has been developed using a cloud-native modern-day architecture.

Aurora Music is an online platform where people can upload, perform and manage music with the use of an easy-to-use interface. In addition, Aurora Music provides audio identification through the Librosa library; thus facilitating the identification of audio files. The front end of Aurora is designed for modern use by providing a dark glassmorphism style and allows users to visualize their audio content through the HTML5 Web Audio API in real time.

The backend application for Aurora is built with the Flask framework using Amazon Web Services (AWS) Docker Containers for deployment. The cloud stack consists of many AWS services and integrates together to form a complete application stack. The services leveraged include: EC2 for compute, RDS for relational database storage, S3 for storing audio files, IAM for access control, ECR for container images, CloudFront for distributing content and CloudWatch for logging and monitoring.

The overall purpose of the Aurora Music project is to show how to implement a cloud architecture that is secure, scalable and production-ready for a music streaming application.

2 Literature Review

Cloud-native applications have become much more prevalent in recent years because they are much more scalable, flexible and enable quicker deployment through distributed computing. For example, Pahl suggests that using PaaS containerization technologies help improve the level of consistency and portability associated with the deployment of applications across different infrastructures [1]. Merkel stated that Docker containers provide lightweight virtualisation technology that allows for simpler deployment of applications dependency management [8].

Modern distributed systems increasingly adopt microservices and cloud-native architectures for improved scalability and maintainability. Balalaie et al. demonstrated how microservices support DevOps practices and enhance modular application development [4]. Dragoni et al. further explained the evolution of microservices and their role in building scalable distributed applications [9].

Cloud computing infrastructure plays a major role in modern multimedia and streaming applications. Buyya et al. introduced the concept of utility-oriented cloud computing and explained how cloud services provide scalable computing resources on demand [11]. Vogels discussed Infrastructure as a Service (IaaS) and highlighted how cloud infrastructure simplifies deployment and resource allocation [15].

Amazon Web Services (AWS) provides managed services that simplify application hosting, storage, and monitoring. Services such as Amazon EC2, Amazon S3, and Amazon RDS are widely used in cloud-native web applications for compute, object storage, and relational database management [5, 6, 7]. Content delivery and caching services such as Amazon CloudFront improve application performance and reduce latency for geographically distributed users.

Research in audio processing and music information retrieval has also grown rapidly. McFee et al. introduced Librosa, a Python library for music and audio signal analysis [3]. The library provides tools for waveform analysis, spectrogram generation, beat tracking, and feature extraction, making it suitable for intelligent music applications and audio recognition systems.

Container orchestration and distributed application deployment are essential for scalable cloud systems. Burns et al. explained how Kubernetes and container orchestration technologies support fault-tolerant and scalable application deployment [2]. These technologies inspired the modular deployment structure used in Aurora Music.

Monitoring and centralized logging systems are critical for cloud-native architectures. Cloud-based logging systems such as Amazon CloudWatch simplify debugging, infrastructure monitoring, and application maintenance. Secure secret management using IAM roles and Systems Manager Parameter Store eliminates hardcoded credentials and improves cloud security practices.

Aurora Music integrates these technologies and research concepts into a unified cloud-native platform for music streaming and audio identification. The project combines distributed cloud services, Docker containerization, secure cloud storage, centralized monitoring, and intelligent audio processing to create a scalable and production-ready application.

3 Proposed Methodology

Aurora Music utilizes a cloud-native infrastructure that consists of several AWS services working cooperatively in a distributed environment.

The frontend user submits requests via Route 53 and CloudFront so that their request can be routed to the application load balancer (ALB). The ALB will then route the request on to the EC2 instance containing the Dockerised Flask application.

The backend portion of the application, built using Flask, manages authentication, photo uploads, and playlist management, as well as user-generated photo uploads for streaming. All uploaded music is stored within Amazon S3 and their associated metadata (customer information, song metadata, playlist information) are stored within Amazon's RDS MySQL Database.

As part of processing uploaded media to be able to identify it later, the application uses several libraries, such as Librosa and FFmpeg, to process the uploaded music file and extract features from it, while the docker application would create a container consisting of all of the components used by the application to ensure consistency and portability across multiple deployment instances.

IAM roles for the EC2 instance provide access to the various S3, ECR, SSM, and CloudWatch services in a secure manner without exposing any credentials; any sensitive configuration values (such as database connection strings) are stored securely in the AWS Systems Manager Parameter Store.

Using CloudWatch Logs, the application will aggregate logs generated from the Docker container instance providing a single location to monitor and debug the application.

3.1 Architecture Diagram

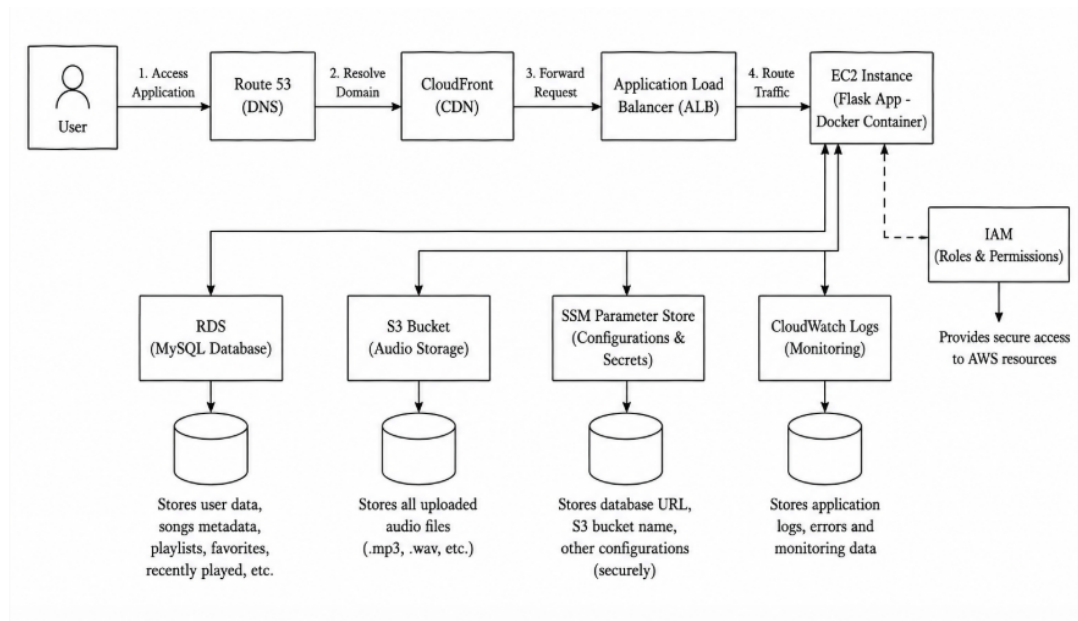


Figure 1: Aurora Music AWS Cloud Architecture

4 Results and Discussion

The Aurora Music project utilized Amazon Web Services through Docker and many other services.

The application successfully supports:

- User registration and login
- Audio upload and storage
- Real-time music streaming
- Favorite song management
- Recently played tracking
- Audio identification using Librosa

Using Amazon S3 means that uploaded audio files can be stored in a reliable and scalable fashion. Amazon RDS allows for the secure and efficient handling of relational data. Docker allowed for consistent deployment across environments, while ECR made it easy to manage the container images.

CloudWatch Logs provided central monitoring for debugging and maintenance purposes. The use of IAM roles and Systems Manager increased security because it eliminated the use of hardcoded credentials.

CloudFront allowed for fast delivery of static content by decreasing latency. The Application Load Balancer allowed for increased resources and allows for future horizontal scaling of the backend servers.

The Architecture used in the project had high levels of modularity, scalability, and security which make this architecture suitable for production use.

5 Conclusion

Aurora Music has successfully built a cloud-based music streaming/audio ID service with Flask, Docker, and AWS. In order to create an architecture that can scale in response to demand and accommodate the changing needs of users, multiple AWS services were used to provide scalable compute resources, secure object storage, relational database management, centralized monitoring, and global content distribution.

The Aurora Music architecture is able to support audio file uploading and streaming, metadata management, and audio file processing while utilizing Docker containers to ensure consistent deployment of all components.

While the current implementation of security best practices (including IAM role-based access and secure secret management via the Systems Manager Parameter Store) is functional, future improvements might implement AI-based music recommendation systems, advanced search algorithms, HTTPS certificate integration, and automatically scaling infrastructure to handle large amounts of user traffic.

References

- [1] B. Pahl, “Containerization and the PaaS Cloud,” *IEEE Cloud Computing*, vol. 2, no. 3, pp. 24–31, 2015.
- [2] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.
- [3] B. McFee et al., “Librosa: Audio and Music Signal Analysis in Python,” *Proceedings of the 14th Python in Science Conference*, pp. 18–25, 2015.
- [4] A. Balalaie, A. Heydarnoori, and P. Jamshidi, “Microservices Architecture Enables DevOps,” *IEEE Software*, vol. 33, no. 3, pp. 42–52, 2016.
- [5] Amazon Web Services, “Amazon EC2 Documentation,” Available: <https://docs.aws.amazon.com/ec2/>
- [6] Amazon Web Services, “Amazon S3 Documentation,” Available: <https://docs.aws.amazon.com/s3/>
- [7] Amazon Web Services, “Amazon RDS Documentation,” Available: <https://docs.aws.amazon.com/rds/>
- [8] D. Merkel, “Docker: Lightweight Linux Containers for Consistent Development and Deployment,” *Linux Journal*, vol. 2014, no. 239, pp. 2–7, 2014.
- [9] N. Dragoni et al., “Microservices: Yesterday, Today, and Tomorrow,” *Present and Ulterior Software Engineering*, pp. 195–216, 2017.
- [10] M. Villamizar et al., “Infrastructure Cost Comparison of Running Web Applications in the Cloud using AWS Lambda and Monolithic and Microservice Architectures,” *IEEE Latin America Transactions*, vol. 15, no. 12, pp. 2332–2340, 2017.
- [11] R. Buyya, C. Yeo, and S. Venugopal, “Market-Oriented Cloud Computing: Vision, Hype, and Reality for Delivering IT Services as Computing Utilities,” *Future Generation Computer Systems*, vol. 25, no. 6, pp. 599–616, 2009.

- [12] M. Fowler and J. Lewis, “Microservices: A Definition of This New Architectural Term,” *ThoughtWorks*, 2014.
- [13] J. Turnbull, *The Docker Book: Containerization is the New Virtualization*, 2014.
- [14] C. Hewitt, “Actor Model of Computation for Scalable Distributed Systems,” *International Conference on Distributed Computing Systems*, 2010.
- [15] W. Vogels, “A Head in the Clouds: The Power of Infrastructure as a Service,” *Queue*, vol. 6, no. 1, pp. 10–22, 2008.